

WILP, BITS Pilani



DEEP NEURAL NETWORKS (AIML ZG511) · COMPANION READER

CNN Patterns, Transfer Learning & Sequence Models

From Spatial Features and ResNets to RNNs, BPTT, GRU, and LSTM Memory Cells

Session 9 (5th July 2026) · Read alongside the lecture slides · Every slide example worked out in full

Prepared for the Work Integrated Learning Programmes (WILP), BITS Pilani.

How to use this companion. This reader expands Session 9 into a complete study guide. We cover two core themes: advanced **Convolutional Neural Network (CNN)** design patterns and **Recurrent Neural Network (RNN)** sequence architectures. Every formula is derived step by step. Every slide example is calculated with real numbers. Use the colour-coded boxes for intuition, everyday analogies, worked math, and common pitfalls.

1 Opening by doing: Spatial features vs temporal memory

Try a 2-step sequence calculation by hand before defining recurrent states.

Consider a small factory line that processes incoming data. An image model looks at a whole grid at once. A sequence model reads items one by one over time. Let us calculate how a simple temporal hidden state updates over 2 steps right now.

Suppose your initial memory state is $h_0 = [0, 0]^T$. At time step $t = 1$, an input $x_1 = [1, 0]^T$ arrives. The memory weight matrix is $W_{hh} = \begin{bmatrix} 0.5 & 0.0 \\ 0.0 & 0.5 \end{bmatrix}$ and input weight matrix is $W_{hx} = \begin{bmatrix} 0.8 & 0.0 \\ 0.0 & 0.8 \end{bmatrix}$.

The raw hidden state before activation is $z_1 = W_{hh}h_0 + W_{hx}x_1 = [0, 0]^T + [0.8, 0.0]^T = [0.8, 0.0]^T$. Applying tanh gives $h_1 = [\tanh(0.8), \tanh(0.0)]^T = [0.664, 0.0]^T$.

At time step $t = 2$, the input changes to $x_2 = [0, 1]^T$. The new state incorporates past memory h_1 :

$$z_2 = W_{hh}h_1 + W_{hx}x_2 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.664 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.8 & 0 \\ 0 & 0.8 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.332 \\ 0.800 \end{bmatrix}. \quad (1)$$

Then $h_2 = [\tanh(0.332), \tanh(0.800)]^T = [0.321, 0.664]^T$.

Notice what just happened: h_2 carries information from *both* x_1 and x_2 . That single matrix multiplication $W_{hh}h_{t-1}$ is the magic ingredient that gives neural networks temporal memory!

2 CNN design patterns and architectural decisions

Five reusable design patterns turn arbitrary layers into scalable, high-performance visual networks.

A typical Convolutional Neural Network alternates between convolution, non-linear activation, and spatial pooling. Early layers capture low-level features such as edges and corners. Middle layers combine edges into textures and shapes. Later layers form high-level concepts like object parts or entire faces. To build deep networks reliably, five key design patterns are used in modern deep learning:

1. **Pattern 1: Spatial Reduction + Channel Expansion.** As spatial dimensions decrease ($32 \times 32 \rightarrow 14 \times 14 \rightarrow 5 \times 5$), feature channels increase ($3 \rightarrow 16 \rightarrow 64 \rightarrow 256$). This preserves computational budget while increasing representational capacity.
2. **Pattern 2: Repetitive Building Blocks.** Replace custom layer arrangements with repeated modular blocks (e.g., VGG blocks or ResNet blocks) for easier scaling.
3. **Pattern 3: Skip / Residual Connections.** Pass inputs directly across layers ($y = F(x) + x$). If a layer learns nothing, it defaults to the identity mapping. This solves vanishing gradients in networks deeper than 100 layers.
4. **Pattern 4: Bottleneck Designs.** Use 1×1 convolutions to squash channel depth before expensive 3×3 convolutions, then expand depth back. This reduces parameters by up to 8x.
5. **Pattern 5: Multi-Scale Feature Extraction.** Use parallel filters of different sizes ($1 \times 1, 3 \times 3, 5 \times 5$) inside Inception modules to capture features at multiple spatial scales simultaneously.

The intuition

A residual block allows gradients to flow down an express highway during backpropagation. Instead of forcing layers to re-learn the full signal, the layer only learns the *change* or residual $F(x)$.

★ Everyday picture

Think of an express train line alongside local tracks. The local tracks ($F(x)$) perform detailed station-by-station work. The express track (x) bypasses stations entirely, letting passenger signals travel directly to the destination without getting lost.

Worked example: Bottleneck vs Standard Convolution Parameter Reduction

Compare a standard convolution layer against a bottleneck block operating on a tensor with 256 channels.

Step 1. Standard Conv 3x3. Input: 256 channels, Output: 256 channels.

$$\text{Parameters} = 3 \times 3 \times 256 \times 256 = 589,824 \approx 590\text{K}.$$

Step 2. Bottleneck Block.

- Step A (1×1 conv to 64 channels): $1 \times 1 \times 256 \times 64 = 16,384$.
- Step B (3×3 conv on 64 channels): $3 \times 3 \times 64 \times 64 = 36,864$.

CNN Bottleneck & Residual Design Patterns

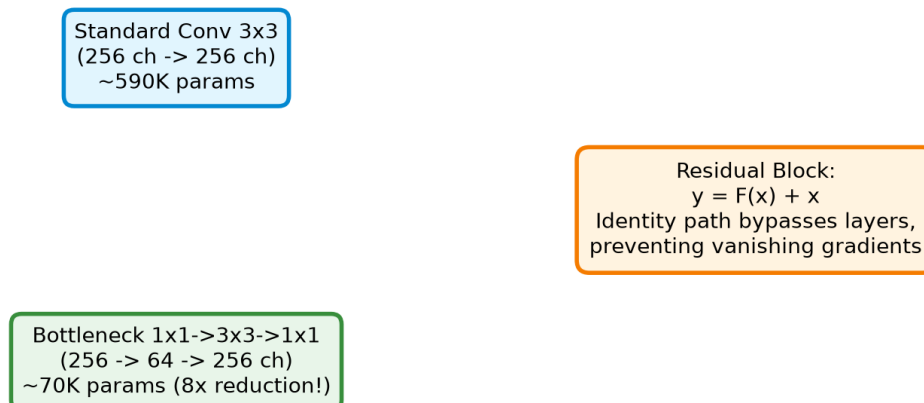


Figure 1: CNN Design Patterns: Spatial reduction, channel expansion, bottleneck parameter reduction, and residual skip connections.

- Step C (1×1 conv to 256 channels): $1 \times 1 \times 64 \times 256 = 16,384$.
Total Parameters = $16,384 + 36,864 + 16,384 = 69,632 \approx 70K$.

Step 3. Reduction ratio. $589,824/69,632 \approx 8.47 \times$ parameter reduction!

✗ Watch out

Depth alone is not enough. Adding layers to a plain feedforward network without skip connections or batch normalization causes training accuracy to degrade rapidly due to vanishing gradients.

✓ Key takeaway

Effective CNN architectures combine spatial reduction with channel expansion, standard modular blocks, residual connections, and bottleneck layers.

3 Classic CNN architectures and transfer learning

AlexNet, VGG, and GoogLeNet laid the groundwork for modern computer vision and transfer learning.

The evolution of ImageNet models highlights key architectural shifts:

- **AlexNet (2012):** 8 layers, 60M parameters. Introduced ReLU, Local Response Normalization (LRN), Dropout (0.5), heavy data augmentation, and GPU training.
- **VGG-16 (2014):** 16 layers, 138M parameters. Proved that stacking simple 3×3 convolutions with 2×2 max pooling yields deeper, richer features.
- **GoogLeNet / Inception (2014):** 22 layers, 5M parameters (12x fewer parameters than AlexNet). Used Inception modules and replaced heavy Fully Connected (FC) layers with Global Average Pooling (GAP).

Table 1: Comparison of Classic Vision Architectures

Architecture	Depth (Layers)	Parameters	Key Innovation	Classifier
AlexNet	8	60M	ReLU, Dropout, LRN	3 FC Layers
VGG-16	16	138M	Small 3×3 filters stacked	3 FC Layers
GoogLeNet	22	5M	Inception parallel blocks	Global Avg Pool
ResNet-50	50	25.6M	Residual skip connections	Global Avg Pool

Transfer Learning Strategy. Instead of training from scratch, pre-trained weights on large datasets (like ImageNet) can be transferred to new target tasks:

1. **Small target dataset + high similarity:** Freeze pre-trained backbone; train linear classifier on top features.
2. **Large target dataset + high similarity:** Fine-tune all layers with a small learning rate.
3. **Small target dataset + low similarity:** Freeze early layers; train higher layers and linear head.
4. **Large target dataset + low similarity:** Train whole network from scratch or fine-tune completely.

✓ Key takeaway

Transfer learning uses low-level feature representations learned from massive datasets, reducing training time and data requirements for specialized tasks.

4 Sequence modeling: Autoregressive vs latent variable models

Standard feedforward networks fail on sequence data because inputs vary in length and lack temporal memory.

Standard feedforward networks assume inputs are independent and identically distributed (i.i.d.) with fixed spatial dimensions. Sequence tasks (e.g., text, speech, stock market, temperature tracking) break these assumptions:

- Inputs and outputs have variable lengths.
- Order matters: "Not good, bad" vs "Good, not bad".
- Temporal parameters must be shared across time steps.

To model sequence data $x_1, x_2, \dots, x_T$, two primary approaches exist:

1. **Autoregressive Models:** Predict current step x_t using a fixed window of past observations:

$$x_t = P(x_t | x_{t-1}, x_{t-2}, \dots, x_{t-\tau}).$$

Under a first-order **Markov Assumption**, $P(x_t | x_{t-1}, \dots, x_1) = P(x_t | x_{t-1})$.

2. **Latent Variable Models (RNNs):** Maintain an internal hidden state h_t that summarizes the entire history up to time t :

$$h_t = g(h_{t-1}, x_t), \quad \hat{y}_t = f(h_t).$$

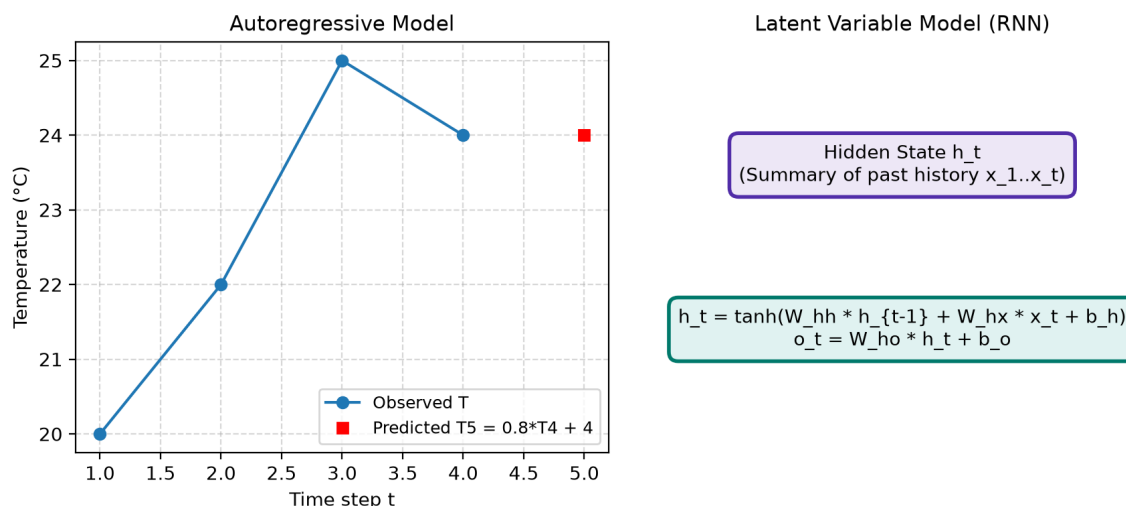


Figure 2: Autoregressive prediction using past observations vs Latent variable models tracking hidden state history.

Worked example: Temperature Forecasting — Autoregressive vs Latent Variable Model

Consider a temperature sequence: 20°C, 22°C, 25°C, 24°C, 21°C. Predict tomorrow's temperature.

Step 1. Autoregressive First-Order Model. Formula: $T_{t+1} = \alpha T_t + \beta$. Given learned

weights $\alpha = 0.8, \beta = 4.0$, and today's temperature $T_5 = 21^\circ\text{C}$:

$$T_{\text{tomorrow}} = 0.8 \times 21 + 4.0 = 16.8 + 4.0 = 20.8^\circ\text{C}.$$

Step 2. Latent Variable Recurrent Model. Formula: $h_t = \tanh(W_{hh}h_{t-1} + W_{hx}T_t + b_h)$. The hidden state h_5 integrates the entire sequence trend ($20 \rightarrow 22 \rightarrow 25 \rightarrow 24 \rightarrow 21$) rather than relying solely on today's single reading.

✓ Key takeaway

Latent variable models compress long sequence histories into a fixed-size hidden state vector h_t , avoiding the need for fixed context windows.

5 Vanilla Recurrent Neural Networks (RNNs)

Unrolling an RNN in time reveals a deep network with parameter sharing across all time steps.

A vanilla Recurrent Neural Network processes input sequence $x_1, x_2, \dots, x_T$ sequentially. At step t , it updates hidden state h_t and computes output o_t :

$$h_t = \phi(W_{hh}h_{t-1} + W_{hx}x_t + b_h), \quad (2)$$

$$o_t = W_{ho}h_t + b_o, \quad (3)$$

$$\hat{y}_t = \text{softmax}(o_t) \quad (\text{or linear for regression}). \quad (4)$$

Here $\phi(\cdot)$ is typically the hyperbolic tangent function ($\tanh$). Parameters $W_{hh}, W_{hx}, W_{ho}, b_h, b_o$ are shared identically across every time step.

Unrolled Vanilla Recurrent Neural Network

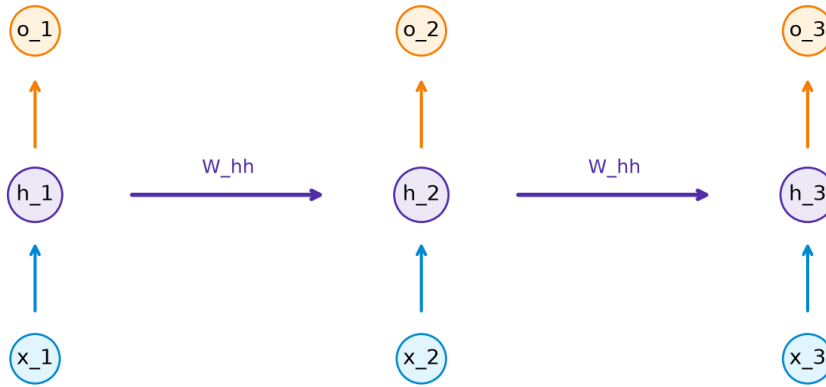


Figure 3: Unrolled RNN computation graph showing shared weight matrices W_{hx}, W_{hh}, W_{ho} across time.

Worked example: RNN Parameter Counting

Calculate total parameters for a vanilla RNN with input dimension $d = 100$, hidden dimension $h = 256$, and vocabulary/output size $V = 10,000$.

Step 1. Hidden layer weight matrices and bias:

- $W_{hx} \in \mathbb{R}^{h \times d}$: $256 \times 100 = 25,600$.
- $W_{hh} \in \mathbb{R}^{h \times h}$: $256 \times 256 = 65,536$.
- $b_h \in \mathbb{R}^h$: 256.

Step 2. Output layer weight matrix and bias:

- $W_{ho} \in \mathbb{R}^{V \times h}$: $10,000 \times 256 = 2,560,000$.
- $b_o \in \mathbb{R}^V$: 10,000.

Step 3. Total parameters:

$$\text{Total} = 25,600 + 65,536 + 256 + 2,560,000 + 10,000 = 2,661,392 \approx 2.66\text{M}.$$

 Worked example: Step-by-Step RNN Forward Pass (2 Time Steps)

Given parameters:

$$W_{hx} = \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \\ 0.1 & 0.2 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.3 & 0.2 & 0.1 \\ 0.1 & 0.4 & 0.2 \\ 0.2 & 0.1 & 0.3 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix},$$

$$W_{ho} = \begin{bmatrix} 0.2 & 0.6 & 0.1 \\ 0.4 & 0.3 & 0.5 \end{bmatrix}, \quad b_o = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}, \quad h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Input sequence: $x_1 = [1.0, 0.5]^T, x_2 = [0.8, 1.2]^T$.

Step 1. Time Step $t = 1$ — Hidden State h_1 :

$$z_1 = W_{hh}h_0 + W_{hx}x_1 + b_h = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5(1.0) + 0.3(0.5) \\ 0.2(1.0) + 0.4(0.5) \\ 0.1(1.0) + 0.2(0.5) \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.50 \\ 0.30 \end{bmatrix}.$$

$$h_1 = \tanh(z_1) = \begin{bmatrix} \tanh(0.75) \\ \tanh(0.50) \\ \tanh(0.30) \end{bmatrix} = \begin{bmatrix} 0.635 \\ 0.462 \\ 0.291 \end{bmatrix}.$$

Step 2. Time Step $t = 1$ — Output o_1 :

$$o_1 = W_{ho}h_1 + b_o = \begin{bmatrix} 0.2(0.635) + 0.6(0.462) + 0.1(0.291) + 0.1 \\ 0.4(0.635) + 0.3(0.462) + 0.5(0.291) + 0.1 \end{bmatrix} = \begin{bmatrix} 0.533 \\ 0.638 \end{bmatrix}.$$

Step 3. Time Step $t = 2$ — Hidden State h_2 :

$$W_{hh}h_1 = \begin{bmatrix} 0.3(0.635) + 0.2(0.462) + 0.1(0.291) \\ 0.1(0.635) + 0.4(0.462) + 0.2(0.291) \\ 0.2(0.635) + 0.1(0.462) + 0.3(0.291) \end{bmatrix} = \begin{bmatrix} 0.312 \\ 0.306 \\ 0.261 \end{bmatrix}.$$

$$W_{hx}x_2 = \begin{bmatrix} 0.5(0.8) + 0.3(1.2) \\ 0.2(0.8) + 0.4(1.2) \\ 0.1(0.8) + 0.2(1.2) \end{bmatrix} = \begin{bmatrix} 0.760 \\ 0.640 \\ 0.320 \end{bmatrix}.$$

$$z_2 = \begin{bmatrix} 0.312 \\ 0.306 \\ 0.261 \end{bmatrix} + \begin{bmatrix} 0.760 \\ 0.640 \\ 0.320 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 1.172 \\ 1.046 \\ 0.681 \end{bmatrix}.$$

$$h_2 = \tanh(z_2) = \begin{bmatrix} \tanh(1.172) \\ \tanh(1.046) \\ \tanh(0.681) \end{bmatrix} = \begin{bmatrix} 0.825 \\ 0.780 \\ 0.592 \end{bmatrix}.$$

Step 4. Time Step $t = 2$ — Output o_2 :

$$o_2 = W_{ho}h_2 + b_o = \begin{bmatrix} 0.2(0.825) + 0.6(0.780) + 0.1(0.592) + 0.1 \\ 0.4(0.825) + 0.3(0.780) + 0.5(0.592) + 0.1 \end{bmatrix} = \begin{bmatrix} 0.792 \\ 0.960 \end{bmatrix}.$$

✓ Key takeaway

A vanilla RNN uses shared weight matrices across time steps to update hidden state
 $h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$.

6 Backpropagation Through Time (BPTT) and vanishing gradients

Unrolling an RNN allows standard backpropagation, but long sequences cause gradients to explode or vanish exponentially.

To train an RNN, loss is calculated across sequence time steps: $L = \sum_{t=1}^T L_t$. **Backpropagation Through Time (BPTT)** calculates gradients with respect to parameters by propagating backwards through time.

The gradient of loss L with respect to hidden state h_k at step $k < T$ involves a chain product of Jacobian matrices:

$$\frac{\partial L_T}{\partial h_k} = \frac{\partial L_T}{\partial h_T} \prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial L_T}{\partial h_T} \prod_{j=k+1}^T \text{diag}(\phi'(z_j)) W_{hh}^T. \quad (5)$$

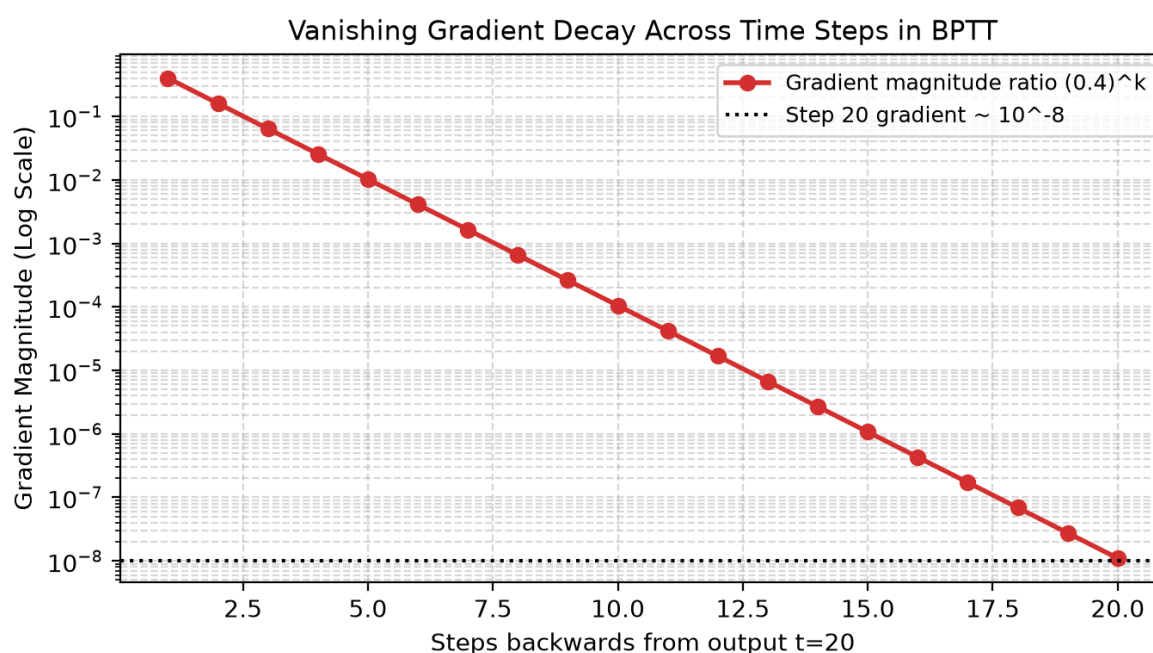


Figure 4: Exponential decay of gradient magnitude across steps in Backpropagation Through Time.

Worked example: Numerical Breakdown of Vanishing Gradients

Suppose the largest eigenvalue of W_{hh} is $\lambda_{\max} = 0.8$, and the activation derivative is $|\phi'(z)| \approx 0.5$.

Step 1. Single Step Multiplier. $\gamma = 0.8 \times 0.5 = 0.4$.

Step 2. After 5 steps ($k = 5$). Gradient ratio = $(0.4)^5 = 0.01024 \approx 1\%$.

Step 3. After 10 steps ($k = 10$). Gradient ratio = $(0.4)^{10} = 0.0001048 \approx 0.01\%$.

Step 4. After 20 steps ($k = 20$). Gradient ratio = $(0.4)^{20} \approx 1.09 \times 10^{-8}$.

Step 5. Conclusion. The gradient signal from step 20 vanishes completely before reaching step 1. Early layers cannot update their parameters to learn long-term context.

✗ Watch out

If weight eigenvalues are greater than 1 (e.g., $\lambda_{\max} > 1.5$), gradients explode exponentially ($1.5^{20} \approx 3325$), causing numerical instability. Gradient clipping is used to cap exploding gradients.

✓ Key takeaway

BPTT backpropagates errors across sequence steps. Repeated matrix multiplication by W_{hh} causes gradients to vanish exponentially for long dependencies.

7 Bidirectional and deep (stacked) RNN architectures

Bidirectional RNNs capture past and future context, while deep RNNs stack hidden layers to build hierarchical representations.

Standard RNNs only use past context $x_1, \dots, x_{t-1}$. In tasks like translation or Named Entity Recognition (NER), future words clarify present meaning (e.g., "I met **Paris** Hilton").

1. **Bidirectional RNN (BRNN):** Runs two separate hidden layers: a forward layer $\vec{h}_t$ ($t = 1 \rightarrow T$) and a backward layer $\overleftarrow{h}_t$ ($t = T \rightarrow 1$). Outputs combine both: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$.
2. **Deep (Stacked) RNN:** Stacks multiple recurrent layers vertically. Layer l receives hidden states from layer $l - 1$ at step t , and hidden states from layer l at step $t - 1$:

$$h_t^{(l)} = \tanh(W_{hh}^{(l)} h_{t-1}^{(l)} + W_{hx}^{(l)} h_t^{(l-1)} + b_h^{(l)}).$$

Worked example: Bidirectional RNN Parameter Counting

Given input $d = 100$, forward hidden $h_f = 128$, backward hidden $h_b = 128$, output $V = 10,000$.

Step 1. Forward RNN Parameters: $128 \times 100 + 128 \times 128 + 128 = 12,800 + 16,384 + 128 = 29,312$.

Step 2. Backward RNN Parameters: $128 \times 100 + 128 \times 128 + 128 = 29,312$.

Step 3. Output Layer Parameters: Concatenated hidden size = $128 + 128 = 256$.

$$W_{ho} \in \mathbb{R}^{10,000 \times 256} \implies 2,560,000, \quad b_o \in \mathbb{R}^{10,000} \implies 10,000.$$

Step 4. Total Parameters: $29,312 + 29,312 + 2,570,000 = 2,628,624 \approx 2.63\text{M}$.

Key takeaway

Bidirectional RNNs process text in both directions to capture full sequence context. Deep RNNs stack recurrent layers to learn abstract hierarchy.

8 Gated Recurrent Unit (GRU)

GRU uses reset and update gates to control information flow, creating additive gradient shortcuts.

To combat vanishing gradients without complex memory structures, the **Gated Recurrent Unit (GRU)** introduces gating mechanisms that selectively retain or forget information over time:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (\text{Reset Gate}), \quad (6)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (\text{Update Gate}), \quad (7)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (\text{Candidate State}), \quad (8)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \quad (\text{Final State}). \quad (9)$$

Gated Recurrent Unit (GRU) Internal Gate Flow

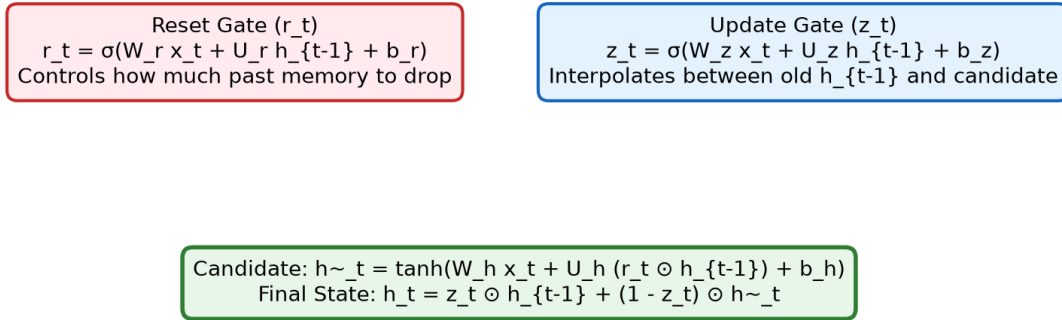


Figure 5: GRU Architecture: Reset gate r_t , update gate z_t , and candidate hidden state $\tilde{h}_t$.

Worked example: Complete 5-Step GRU Numerical Calculation

Inputs: $d = 2, h = 2$. Given $x_1 = [1.0, 0.5]^T, h_0 = [0.6, 0.4]^T$. Weights:

$$\begin{aligned}
 W_r &= \begin{bmatrix} 0.3 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}, U_r = \begin{bmatrix} 0.5 & 0.1 \\ 0.2 & 0.3 \end{bmatrix}, b_r = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}, \\
 W_z &= \begin{bmatrix} 0.2 & 0.5 \\ 0.4 & 0.1 \end{bmatrix}, U_z = \begin{bmatrix} 0.1 & 0.4 \\ 0.3 & 0.2 \end{bmatrix}, b_z = \begin{bmatrix} 0.2 \\ 0.2 \end{bmatrix}, \\
 W_h &= \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \end{bmatrix}, U_h = \begin{bmatrix} 0.4 & 0.2 \\ 0.1 & 0.3 \end{bmatrix}, b_h = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}.
 \end{aligned}$$

Step 1. Step 1: Reset Gate r_1 .

$$W_r x_1 + U_r h_0 + b_r = \begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.34 \\ 0.24 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.84 \\ 0.64 \end{bmatrix},$$

$$r_1 = \sigma \begin{bmatrix} 0.84 \\ 0.64 \end{bmatrix} = \begin{bmatrix} 0.6985 \\ 0.6547 \end{bmatrix}.$$

Step 2. Step 2: Update Gate z_1 .

$$W_z x_1 + U_z h_0 + b_z = \begin{bmatrix} 0.45 \\ 0.45 \end{bmatrix} + \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} + \begin{bmatrix} 0.2 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.87 \\ 0.91 \end{bmatrix},$$

$$z_1 = \sigma \begin{bmatrix} 0.87 \\ 0.91 \end{bmatrix} = \begin{bmatrix} 0.7047 \\ 0.7130 \end{bmatrix}.$$

Step 3. Step 3: Gated Memory $r_1 \odot h_0$.

$$r_1 \odot h_0 = \begin{bmatrix} 0.6985 \times 0.6 \\ 0.6547 \times 0.4 \end{bmatrix} = \begin{bmatrix} 0.4191 \\ 0.2619 \end{bmatrix}.$$

Step 4. Step 4: Candidate Hidden State $\tilde{h}_1$.

$$W_h x_1 + U_h (r_1 \odot h_0) + b_h = \begin{bmatrix} 0.65 \\ 0.40 \end{bmatrix} + \begin{bmatrix} 0.2200 \\ 0.1205 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.9700 \\ 0.6205 \end{bmatrix},$$

$$\tilde{h}_1 = \tanh \begin{bmatrix} 0.9700 \\ 0.6205 \end{bmatrix} = \begin{bmatrix} 0.7487 \\ 0.5516 \end{bmatrix}.$$

Step 5. Step 5: Final Hidden State Update h_1 .

$$h_1 = z_1 \odot h_0 + (1 - z_1) \odot \tilde{h}_1$$

$$= \begin{bmatrix} 0.7047(0.6) + (0.2953)(0.7487) \\ 0.7130(0.4) + (0.2870)(0.5516) \end{bmatrix} = \begin{bmatrix} 0.6439 \\ 0.4435 \end{bmatrix}.$$

Worked example: GRU Parameter Count Formula

For input d and hidden size h , GRU has 3 gate components $(r, z, \tilde{h})$, each with matrices $W \in \mathbb{R}^{h \times d}, U \in \mathbb{R}^{h \times h}, b \in \mathbb{R}^h$.

$$\text{Parameters} = 3 \times (h \cdot d + h^2 + h) + V \cdot h + V.$$

For $d = 100, h = 256, V = 10,000$:

$$\begin{aligned} \text{Recurrent Params} &= 3 \times (256 \times 100 + 256^2 + 256) \\ &= 3 \times (25,600 + 65,536 + 256) = 274,176 \approx 274\text{K}. \end{aligned}$$

Key takeaway

GRU uses an update gate z_t to create an additive shortcut $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$, allowing gradients to pass unchanged across long distances when $z_t \approx 1$.

9 Long Short-Term Memory (LSTM) networks

LSTM separates long-term cell state memory c_t from short-term hidden output h_t using three dedicated gates.

To maintain precise, long-range temporal dependencies over hundreds of steps, the **Long Short-Term Memory (LSTM)** architecture splits state memory into two distinct vectors:

1. **Cell State (c_t):** The long-term memory conveyor belt.
2. **Hidden State (h_t):** The short-term working output.

LSTM operates via three dedicated gates:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (\text{Forget Gate — what to discard}), \quad (10)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (\text{Input Gate — what to update}), \quad (11)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (\text{Candidate Cell State}), \quad (12)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Cell State Update}), \quad (13)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (\text{Output Gate — what to expose}), \quad (14)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{Final Hidden Output}). \quad (15)$$

LSTM Memory Cell Architecture: Long-Term vs Short-Term Flow

Cell State (c_t) Highway: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

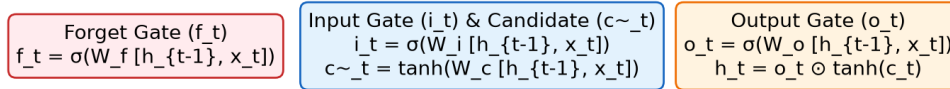


Figure 6: LSTM Architecture: Cell state c_t highway with Forget gate f_t , Input gate i_t , and Output gate o_t .

Worked example: LSTM Parameter Count Formula

LSTM contains 4 gate components ($f, i, o, \tilde{c}$), each with weight matrices $W \in \mathbb{R}^{h \times d}, U \in \mathbb{R}^{h \times h}, b \in \mathbb{R}^h$.

$$\text{Parameters} = 4 \times (h \cdot d + h^2 + h) + V \cdot h + V.$$

For $d = 100, h = 256, V = 10,000$:

$$\begin{aligned} \text{Recurrent Params} &= 4 \times (256 \times 100 + 256^2 + 256) \\ &= 4 \times (25,600 + 65,536 + 256) = 365,568 \approx 366\text{K}. \end{aligned}$$

*** Wait — really?**

In an LSTM cell state update $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, if $f_t = 1$ and $i_t = 0$, the cell state $c_t = c_{t-1}$ remains completely unchanged without *any* decay! Gradients flow backward along $\frac{\partial c_t}{\partial c_{t-1}} = 1.0$ infinitely without vanishing. This constant error carousel is why LSTMs can bridge gaps of over 100 time steps easily.

✓ Key takeaway

LSTM isolates long-term memory inside cell state c_t . It uses forget, input, and output gates to achieve constant error flow across long time horizons.

10 Comparative synthesis, self-test & pocket card

Summary decision rules, rapid self-test, and the complete lecture on a single pocket card.

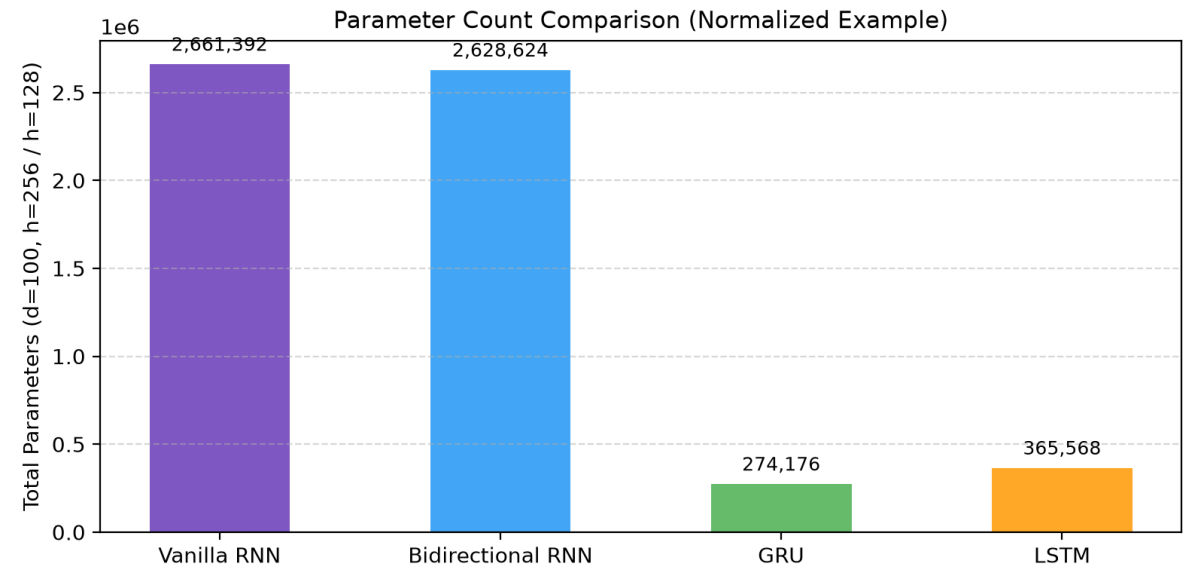


Figure 7: Parameter count comparison across recurrent architecture variants.

Table 2: Recurrent Architecture Selection Matrix				
Architecture	Gates	States	Recurrent Params (h=256, d=100)	Best Use Case
Vanilla RNN	0	h_t	$65.5K + 25.6K = 91.3K$	Short sequences ($T < 10$)
BRNN	0	$\vec{h}_t, \overleftarrow{h}_t$	182.7K	Full sequence context (NER, Classification)
GRU	2 (r, z)	h_t	274.2K	Medium-to-long sequences, fast training
LSTM	3 (f, i, o)	c_t, h_t	365.6K	Complex long dependencies ($T > 100$)

Rapid Self-Test

1. Q1: Why does a 1×1 convolution in a bottleneck block reduce parameters?

A1: It reduces channel depth before expensive 3×3 convolutions, saving up to 8x parameters.
2. Q2: What is the primary difference between autoregressive models and latent variable models?

A2: Autoregressive models use a fixed window of past inputs; latent variable models track state history h_t .
3. Q3: Why does GRU train faster than LSTM?

A3: GRU has 2 gates instead of 3, merged state h_t , and 25% fewer parameters.

✓ Key takeaway

POCKET CARD: Session 9 on One Page

- **CNN Patterns:** Spatial reduction + channel expansion, residual shortcuts ($y = F(x) + x$), bottlenecks (1×1), multi-scale Inception.
- **Classic CNNs:** AlexNet (ReLU/Dropout, 60M), VGG-16 (3×3 convs, 138M), GoogLeNet (Inception, GAP, 5M).
- **Transfer Learning:** Freeze early features for small/similar datasets; fine-tune for

large/different datasets.

- **Vanilla RNN:** $h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$. Shared weights across time cause vanishing gradients ($\prod W_{hh}^T$).
- **BPTT:** Unrolls sequence across time to sum parameter gradients.
- **GRU:** r_t (reset), z_t (update), $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$. $3(hd + h^2 + h)$ params.
- **LSTM:** f_t (forget), i_t (input), o_t (output), cell state c_t . $4(hd + h^2 + h)$ params. Solves vanishing gradient via constant error carousel.

Glossary & symbol cheat-sheet

Key terms.

Bottleneck Block

A $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ conv sequence reducing channel dimensionality to save parameters.

Residual Connection

A skip connection adding input directly to output ($y = F(x) + x$) to preserve gradient flow.

BPTT

Backpropagation Through Time; unrolling an RNN across time steps to sum gradients.

Vanishing Gradient

Exponential decay of gradient magnitudes when backpropagating through long time steps.

Update Gate (z_t)

GRU gate controlling the balance between past hidden state and new candidate state.

Forget Gate (f_t)

LSTM gate deciding what percentage of past cell state memory to erase.

Symbols at a glance.

Symbol	Meaning
h_t	Hidden state vector at sequence time step t
c_t	Cell state vector (long-term memory) in LSTM
W_{hh}, W_{hx}, W_{ho}	Shared weight matrices for hidden-to-hidden, input-to-hidden, hidden-to-output
r_t, z_t	Reset gate and Update gate vectors in GRU
f_t, i_t, o_t	Forget gate, Input gate, and Output gate vectors in LSTM
$\sigma(z)$	Sigmoid activation function squashing values to $[0, 1]$
$\tanh(z)$	Hyperbolic tangent activation squashing values to $[-1, 1]$